

Using CSV File Source for Parameterized Tests

- **JUnit** provides `@CsvFileSource` to load arguments from an external file.
- Useful when your **test needs many input combinations**.
- Helps **avoid** overly **long** or **unreadable inline** `@CsvSource` annotations.
- Keeps **test classes clean** and **easier** to **maintain**.

 **by Rodrigo Martins Pagliares**

Last update: 11/27/2025



Creating the Test Resources Folder (Older versions of IntelliJ)

01

Create a resources folder under the test directory.

02

Mark it as *Test Resources Root* in the IDE's module settings.

03

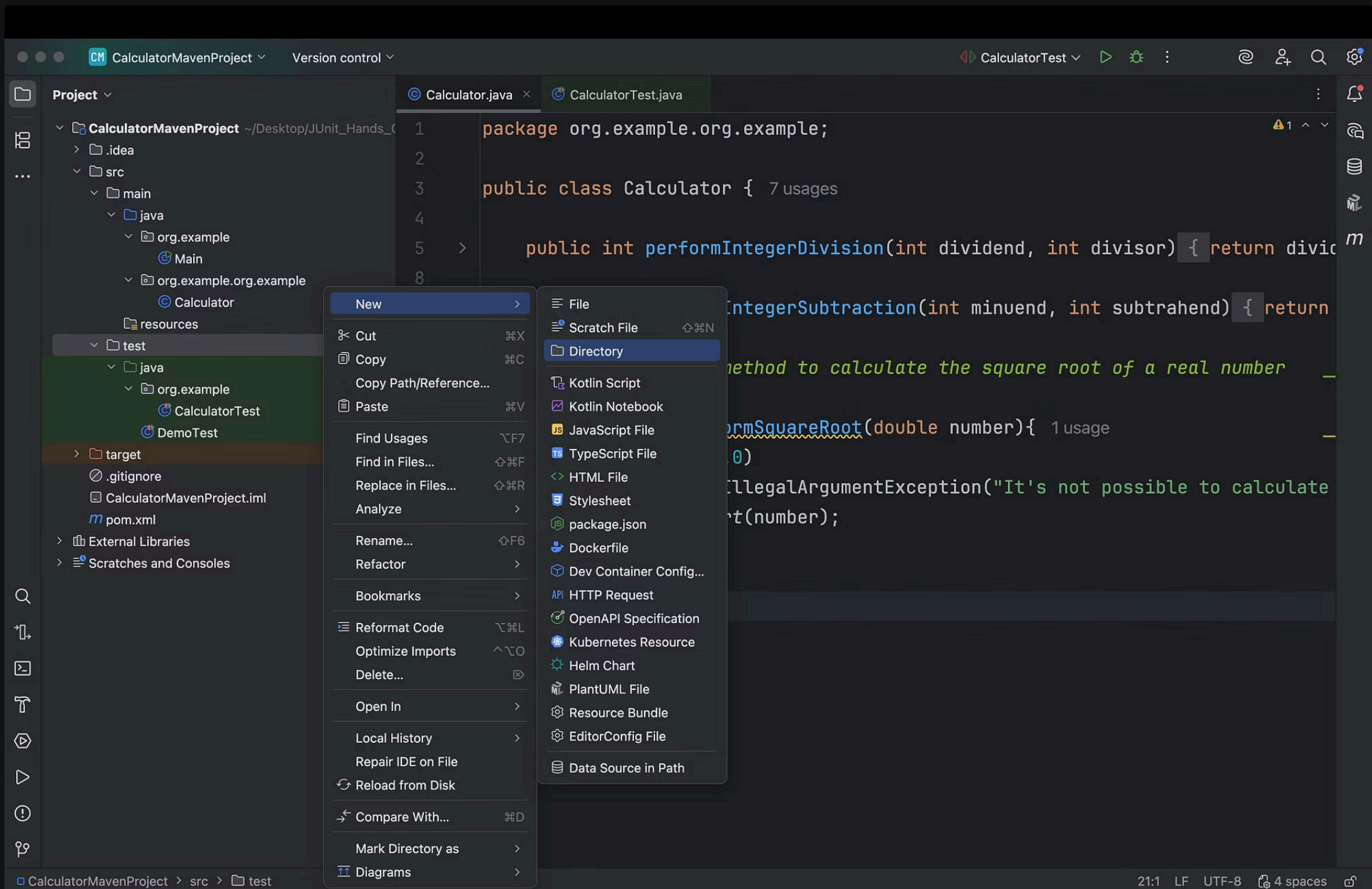
The folder icon changes, indicating correct configuration.

04

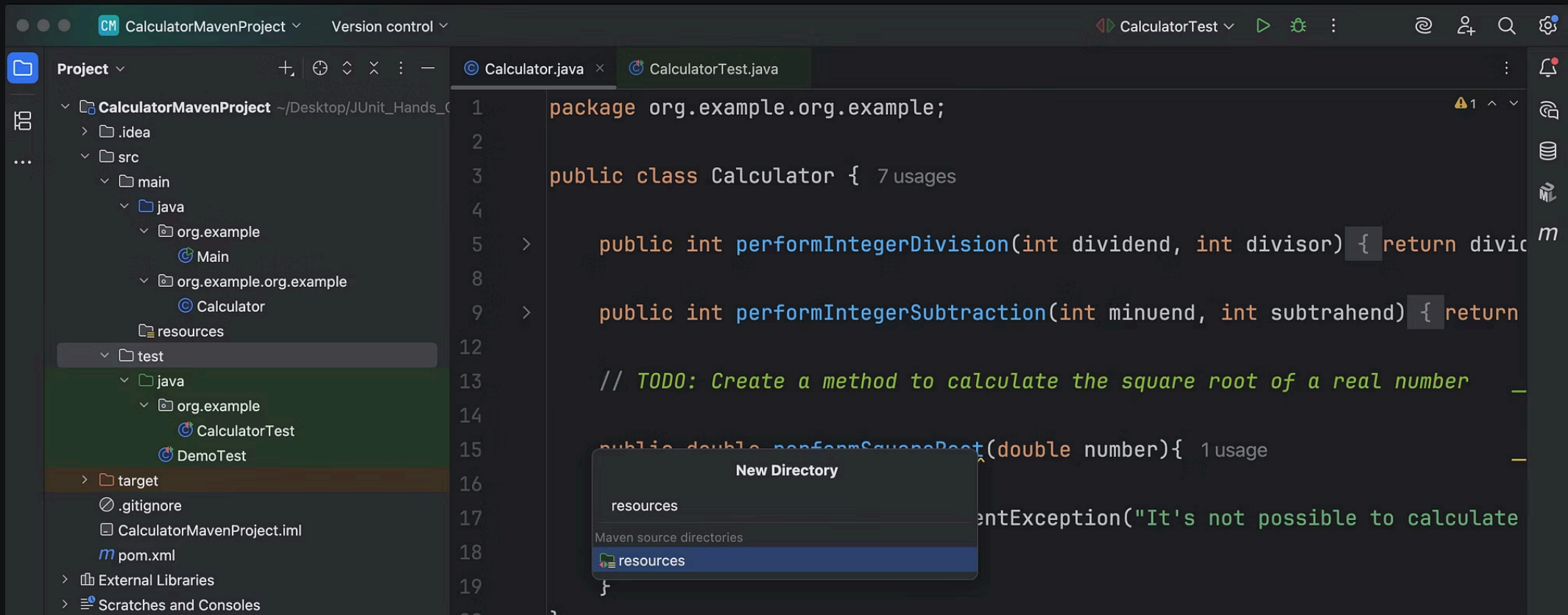
This folder will hold CSV files consumed by parameterized tests.



Creating the Test Resources Folder - IntelliJ IDEA 2025.2.5 (Ultimate Edition)

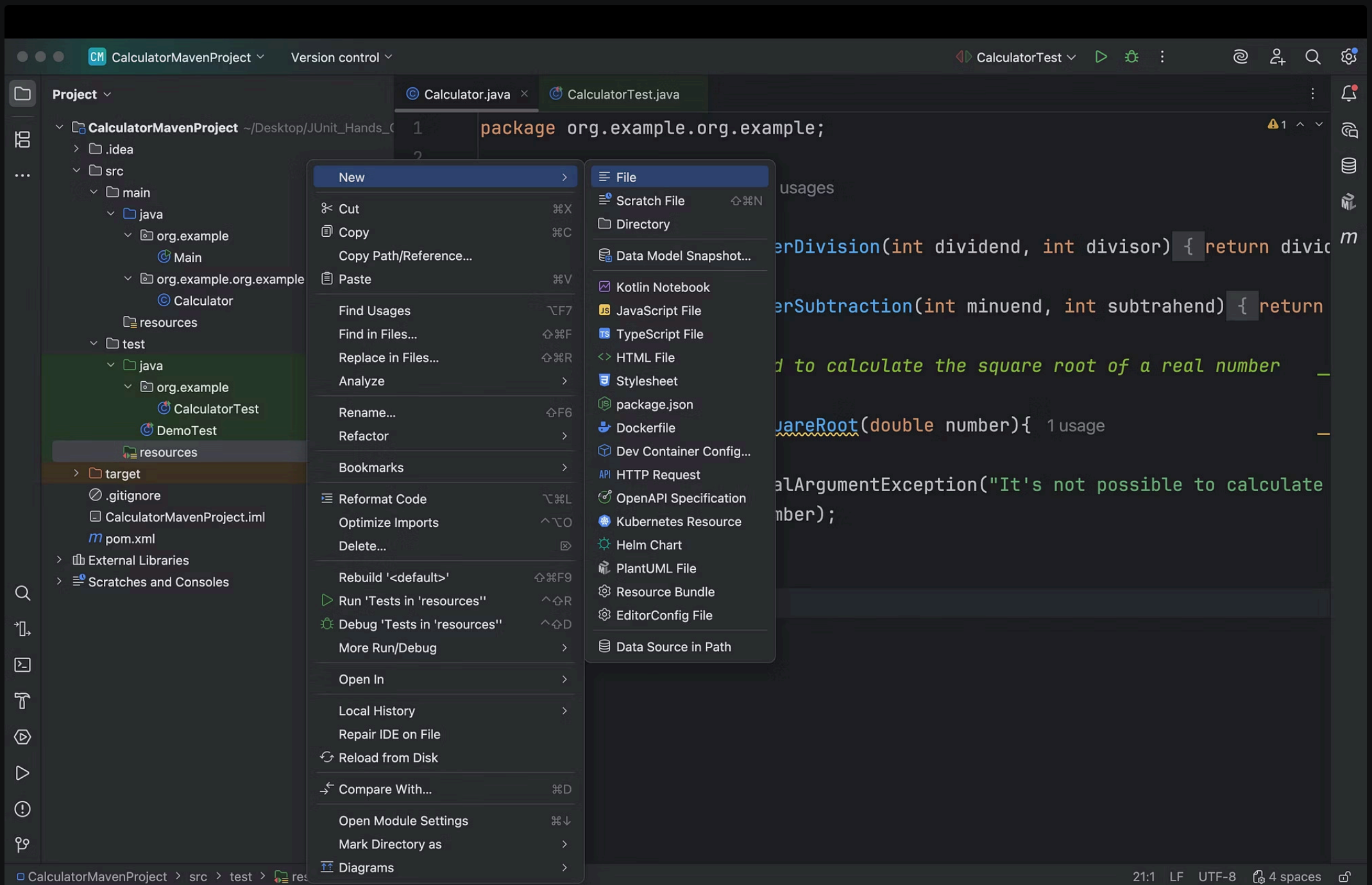


Creating the Test Resources Folder - IntelliJ IDEA 2025.2.5 (Ultimate Edition)



This version of IntelliJ automatically detects the new directory as a test resource folder.

Creating and populate the CSV file in the Test Resources Folder



Name the file:

test_cases_integer_subtraction.csv

Populating the CSV File

- Add **each parameter set** on its own **line, comma-separated**.
- Example lines:
 - 113, 13, 100
 - 15, 15, 0
 - 23, 24, -1,
 - 0, 0, 0
 - 0, 13, -13
- **JUnit** maps these **values** to the **test method parameters** automatically.



Populating the CSV File

Calculator.java		test_cases_integer_subtraction.csv		CalculatorTest.java	
1		113,	13, 100		
2		15,	15, 0		
3		23,	24, -1		
4		0,	0, 0		
5		0,	13, -13		

Replacing Inline CSV with File-Based Input

- **Comment out** or remove the `@CsvSource` annotation.
- Add `@CsvFileSource(resources = "/test_cases_integer_subtraction.csv")`.
- The **resources path** points to a **file** inside the test resources folder.
- **File name** must **match** exactly the one **referenced** in the **annotation**.



Replacing Inline CSV with File-Based Input

```
@CsvFileSource(resources = "/test_cases_integer_subtraction.csv")
```

```
56     @DisplayName("Test integer subtraction (minuend, subtrahend, expectedResult)")
57     @ParameterizedTest
58     // @MethodSource("generateInputParametersForIntegerSubtractionTestMethod")
59     //     @CsvSource({
60     //         "113, 13, 100",
61     //         "15, 15, 0",
62     //         "23, 24, -1",
63     //         "0, 0, 0",
64     //         "0, 13, -13"
65     //     })
66     @CsvFileSource(resources = "/test_cases_integer_subtraction.csv")
67     public void testPerformIntegerSubtraction(
68         int minuend, int subtrahend, int expectedResult) {
```

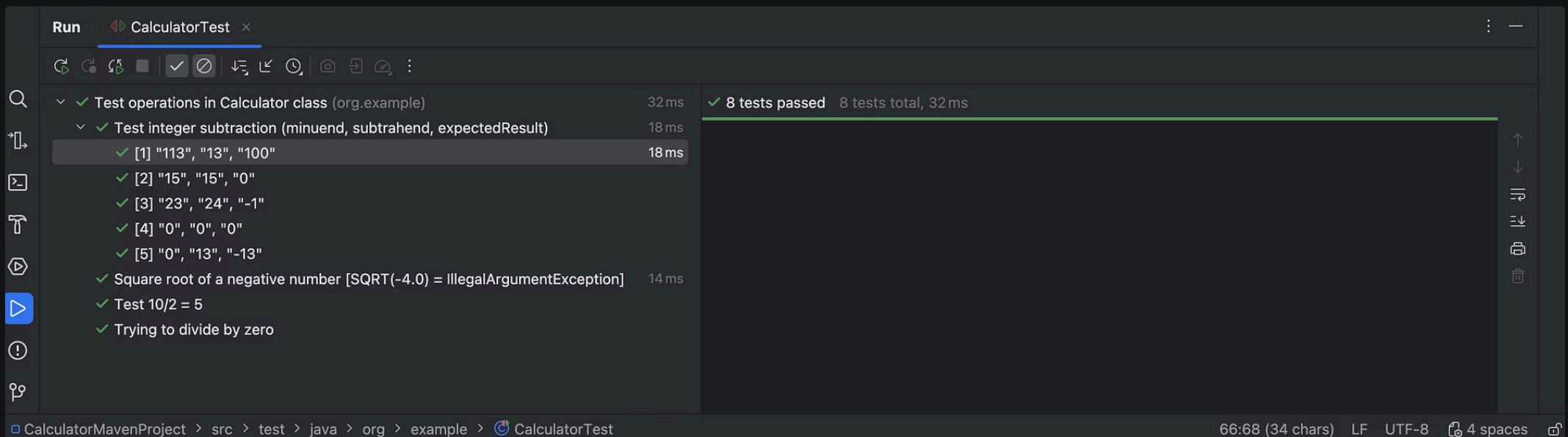
Running Tests with CSV File Source

After saving the file, run the parameterized test normally.

Each line in the CSV file triggers one test execution.

IDE automatically detects and loads the CSV from the test resources folder.

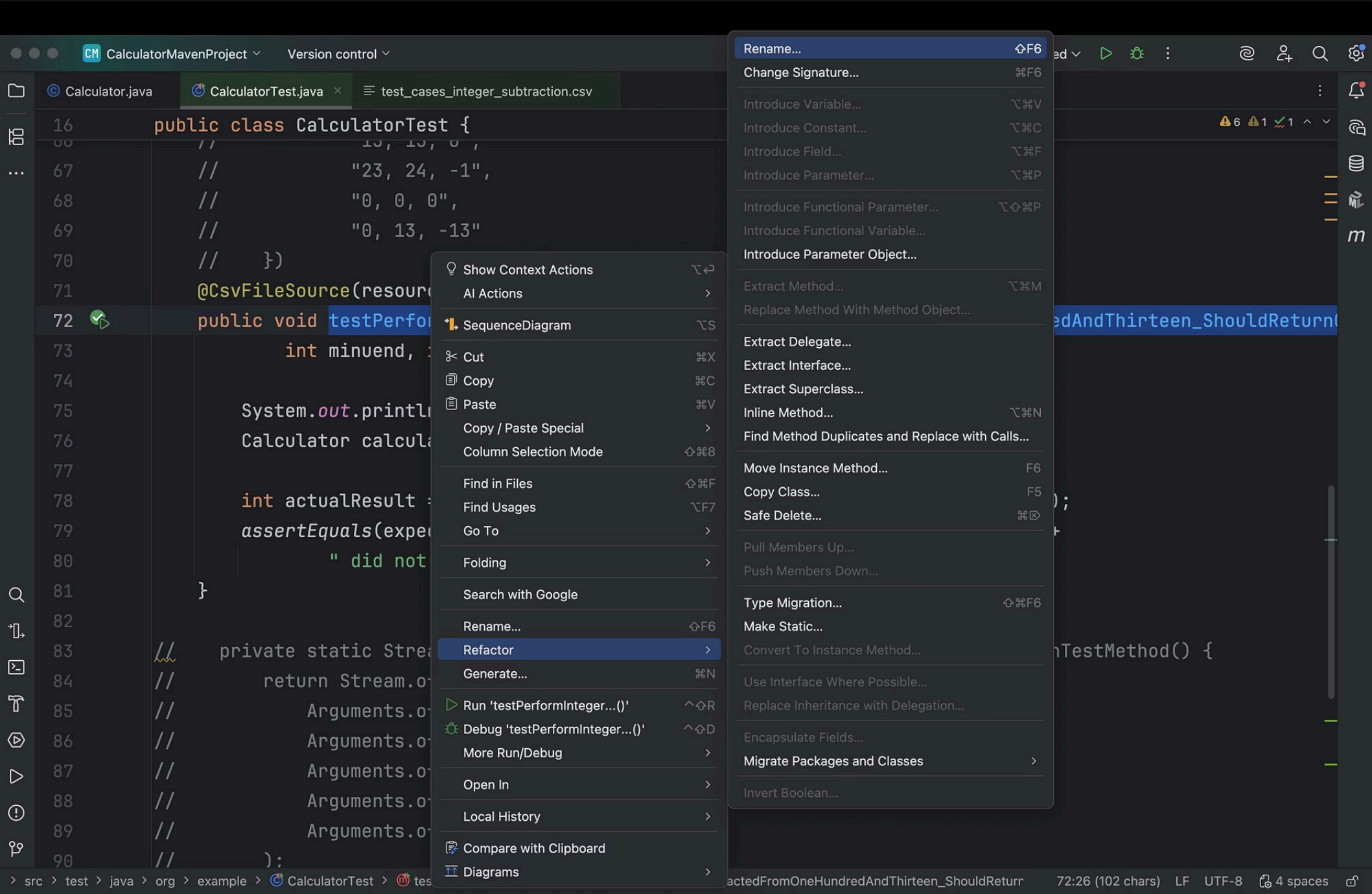
Test results should show one execution per data row.



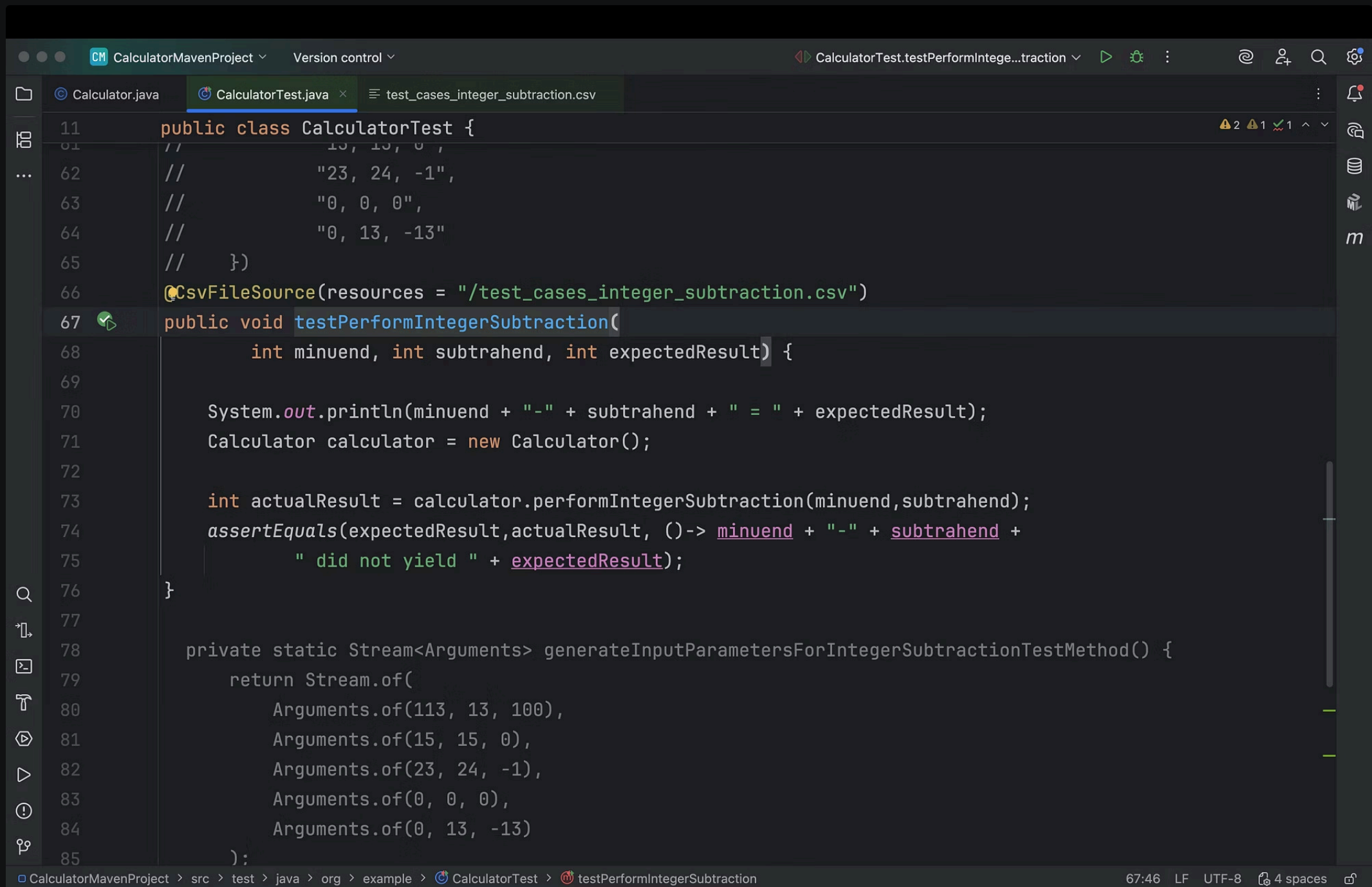
Refactor: Rename method

Since our test method now receives multiple input parameters, we can rename its name to a more generic name, even that breaking ou pattern:

test<method>_<condition>_<expectedResult>



Refactor: Rename method (contd.)



The screenshot shows an IDE window for a project named 'CalculatorMavenProject'. The active file is 'CalculatorTest.java'. The code defines a 'CalculatorTest' class with a 'testPerformIntegerSubtraction' method. The method signature is 'public void testPerformIntegerSubtraction(int minuend, int subtrahend, int expectedResult)'. The method body includes a print statement, a 'new Calculator()' instantiation, and an 'assertEquals' call. A static method 'generateInputParametersForIntegerSubtractionTestMethod()' is also present, returning a stream of arguments. The IDE interface includes a sidebar with file explorer, search, and other tools. The bottom status bar shows the file path 'CalculatorMavenProject > src > test > java > org > example > CalculatorTest > testPerformIntegerSubtraction' and settings like '67:46 LF UTF-8 4 spaces'.

```
11 public class CalculatorTest {
12     // 13, 13, 0,
13     // 23, 24, -1,
14     // 0, 0, 0,
15     // 0, 13, -13
16     // })
17     @CsvFileSource(resources = "/test_cases_integer_subtraction.csv")
18     public void testPerformIntegerSubtraction(
19         int minuend, int subtrahend, int expectedResult) {
20
21         System.out.println(minuend + "-" + subtrahend + " = " + expectedResult);
22         Calculator calculator = new Calculator();
23
24         int actualResult = calculator.performIntegerSubtraction(minuend, subtrahend);
25         assertEquals(expectedResult, actualResult, () -> minuend + "-" + subtrahend +
26             " did not yield " + expectedResult);
27     }
28
29     private static Stream<Arguments> generateInputParametersForIntegerSubtractionTestMethod() {
30         return Stream.of(
31             Arguments.of(113, 13, 100),
32             Arguments.of(15, 15, 0),
33             Arguments.of(23, 24, -1),
34             Arguments.of(0, 0, 0),
35             Arguments.of(0, 13, -13)
36         );
37     }
38 }
```

Benefits of External CSV Files



Supports very large input lists (names, emails, IDs, etc.).



Keeps test annotations short and clean.



Allows updating test data without modifying Java code.



Useful for validating methods against diverse or massive datasets.



Conclusion

- `@CsvFileSource` is ideal for managing **large sets of input data**.
- **Clean separation** between **test logic** and **test data** improves maintainability.
- Easy to **scale testing** by simply **expanding** the **external CSV file**.
- Ensures **consistent execution** of **multiple parameter sets**.

Open Floor Discussion

When would you choose @CsvSource instead of @CsvFileSource?

How can external files improve collaboration between developers and testers?

Could this approach support non-technical contributors generating test data?

